“Software Engineering”
Course
a.a. 2025-2026

Lecturer: Prof. Henry Muccini (henry.muccini@univaq.it)

Garbage Collector

Data	<31/08/2026>
Deliverable	Deliverable – D3
Team (Nome)	Git Gud

Membri del Team		
Nome e Cognome	**Numero di Matricola**	**Indirizzi e-mail**
Alessia De Dominicis	*295438*	*alessia.dedominicis@student.univaq.it*
Riccardo D’Aviero	*271770*	*riccardo.daviero@student.univaq.it*

Indice del Deliverable

Sommario

A. Contesto (D1) **3**
B. Lista dei Requisiti o Task che costituiscono un Rischio o una Challenge (D1) **5**
C. Stato dell'Arte (D1) **8**
D. Raffinamento dei Requisiti **11**
D.1 Servizi (con prioritizzazione) (D1) 11
D.2 Use Case Diagrams (D1) 14
D.3 Requisiti non Funzionali (D1) 17
E. Architettura Software **19**
E.1 Vista statica del sistema: Component Diagram (D2.A) 19
E.2 Vista dinamica dell'architettura software: Sequence Diagram (D2.A) 22
E.3 Vista dinamica del sistema: State Diagram (D2.B) 26
F. Design di basso livello (D2.B) **28**
F.1 Class Diagram 28
Appendice A. Effort Recording (D1, D2.A, D2.B) **30**

A. Contesto (D1)

Dopo aver analizzato la traccia del progetto, riportare in questa sezione gli elementi chiave dell'applicazione che volete realizzare, ponendo l'accento sulle differenze rispetto alla traccia data. Si consiglia di scrivere questa sezione subito prima della consegna del D1.

Cosa verrà valutato: chiarezza della descrizione del problema e delle differenze rispetto

Garbage Collector
Il progetto Garbage Collector è un sistema informativo per la gestione avanzata dei rifiuti urbani e della raccolta differenziata. Il software si rivolge sia ai cittadini che agli operatori degli enti comunali, con l'obiettivo di migliorare la comunicazione tra amministrazione e cittadinanza, rendere più semplice la gestione dei rifiuti urbani e promuovere consapevolezza tra i cittadini.

Servizi di base (in linea con la traccia)
Il software permette ai cittadini, tramite un'applicazione mobile, di accedere ad una serie di servizi base: visualizzare il calendario della raccolta porta a porta, ricevere promemoria push sul conferimento dei rifiuti, visualizzare su mappa i centri di raccolta per rifiuti speciali/ingombranti e inviare segnalazioni geolocalizzate con documentazione fotografica per comunicare disservizi e abbandono di rifiuti.

Dall'altro lato, il software permette agli operatori comunali, tramite dashboard web, di visualizzare dati aggregati e statistiche sulla raccolta differenziata, aggiornare lo stato delle segnalazioni aperte e pianificare percorsi ottimizzati per i mezzi di raccolta.

Elementi di innovazione (differenze con la traccia)
Attraverso l'analisi dei servizi base menzionati nella traccia, abbiamo individuato una serie di Challenges e Risks per l'architettura e l'affidabilità del software. Abbiamo quindi espanso e raffinato i servizi di base:

- Integrazione di Routing Dinamico tramite API esterne per la pianificazione dei percorsi dei mezzi, che prende in considerazione le segnalazioni degli utenti e le telemetrie sullo stato di riempimento dei cassonetti inviate da sensori montati direttamente al loro interno, tenendo conto anche di ipotetici guasti;
- Rafforzamento della sicurezza del sistema contro bot e spam, implementando l'autenticazione tramite SPID/CIE, sistemi di timeout e prevenzione di segnalazioni duplicate;

- Implementazione di un sistema AI per offuscare i dati sensibili presenti nelle foto allegate alle segnalazioni, come volti e targhe, allo scopo di tutelare i dati personali e conformarsi alle leggi in vigore sulla privacy;
- Realizzazione delle interfacce rispettosa delle direttive WCAG (Web Content Accessibility Guidelines), con integrazione di font scalabili, contrasti elevati e supporto per screen reader, così che il servizio non escluda le persone anziane o ipovedenti.

B. Lista dei Requisiti o Task che costituiscono un Rischio o una Challenge (D1)

Dopo un'accurata analisi del sistema in studio, avendone identificate le criticità, inserire qui tutti quei requisiti o tasks che si ritengono challenging o risky. Inserirne anche una descrizione, e la relativa soluzione. Ogni challenge o rischio deve avere ID come riportato in tabella.

Cosa verrà valutato: descrizione del rischio/challenge accurata, soluzione riportata con eventuale tracciabilità su requisiti o altro.

Challenging/Risky Requisito/Task ID	Elemento	Tipo	Data identificazione	Data risoluzione
C1 - Routing dinamico dei percorsi	RF8, RNF4	Challenge	16/08/2026	17/08/2026
C2 - Controllo e ridondanza delle segnalazioni	RF4, RF5, RNF5	Challenge	17/08/2026	18/08/2026
R1 - Potenziale spam delle segnalazioni	RF1, RNF1, RNF12	Risk	17/08/2026	17/08/2026
R2 - Implementazione IA per oscuramento dati e conformità Privacy	RF6, RNF2	Risk	17/08/2026	19/08/2026
R3 - Assenza di segnale durante l'utilizzo dell'app	RF4, RF11, RNF5	Risk	17/08/2026	18/08/2026
R4 - Perdita di connettività dei Sensori	RF7, RF8, RNF4	Risk	17/08/2026	18/08/2026
R5 - Rischio di escludere anziani e ipovedenti	SOTA1, SOTA2, RF2, RF3, RNF3	Risk	17/08/2026	17/08/2026

- **C1 - Routing dinamico dei percorsi**
  Descrizione: il calcolo dei percorsi più efficienti per i mezzi di raccolta deve considerare diversi fattori: la capacità massima dei camion, gli orari di lavoro degli operatori, lo stato di riempimento dei cassonetti in tempo reale e le segnalazioni di abbandono di rifiuti. Tutto questo rende il problema molto complesso e pesante per il backend.
  Soluzione: Integrazione e utilizzo di API esterne dedicate al calcolo dei percorsi. Uno strumento come la Google Maps Route Optimization API può essere utilizzato per delegare il calcolo più impegnativo.

- **C2 - Controllo e ridondanza delle segnalazioni**
  Descrizione: Gli utenti potrebbero falsare le segnalazioni caricando foto e posizione errate. Inoltre, più utenti potrebbero segnalare lo stesso disservizio o abbandono di rifiuti nonostante questo sia già stato preso in carico.
  Soluzione: L'utente dovrà necessariamente scattare la foto in app senza possibilità di importare la foto dalla galleria, e dovrà inoltre abilitare la geolocalizzazione per allegare la posizione corrente alla segnalazione. Se una segnalazione è stata già aperta nelle vicinanze della posizione rilevata, questa verrà mostrata all'utente che può decidere se confermare la precedente o aprirne un'altra.

- **R1 - Potenziale Spam delle Segnalazioni**
  Descrizione: Rischio di abuso del sistema delle segnalazioni da parte di utenti malintenzionati, creando e utilizzando account falsi o bot esterni per lo spam delle segnalazioni, che rischierebbe di intasare i sistemi.
  Soluzione: Gli utenti possono creare account solo dopo aver effettuato la verifica dell'identità tramite SPID/CIE. In più, dopo ogni segnalazione da parte di un utente, partirà un timeout entro il quale un utente non può effettuare altre segnalazioni.

- **R2 - Conformità linee guida Privacy**
  Descrizione: Nel momento in cui un utente scatta una foto per inviare una segnalazione, la foto potrebbe contenere dati sensibili come volti di passanti o targhe di veicoli o dati personali all'interno dei rifiuti stessi (come bollette e ricevute) che sono protetti dal Regolamento Europeo sulla Privacy (GDPR) e che non potrebbero essere né salvati né divulgati sull'app.
  Soluzione: Implementazione lato Server di una pulizia delle immagini prima che queste vengano salvate nel database con l'utilizzo di un modello IA pre-addestrato che rileva automaticamente volti, targhe e dati sensibili, e che applica una sfocatura per oscurarli. Prima dello scatto della foto verranno comunque mostrate a schermo degli avvisi che consigliano all'utente di non inquadrare dati sensibili.

- **R3 - Assenza di segnale durante l'utilizzo dell'app**
  Descrizione: L'utente dovrebbe poter accedere alle features essenziali dell'app anche in assenza di connettività alla rete. Ad esempio, l'utente deve sempre poter visualizzare la posizione delle aree ecologiche più vicine e deve poter fare una segnalazione anche nelle zone dove la connessione è assente.
  Soluzione: L'app chiede all'utente se vuole scaricare localmente nella memoria del dispositivo le mappe, per poterle visualizzare anche in assenza di connessione. Inoltre, le segnalazioni inviate nelle sessioni offline verranno salvate dall'app all'interno della cache locale e verranno inviate automaticamente una volta che il segnale sarà tornato.

- **R4 - Perdita di connettività dei sensori**
  Descrizione: I sensori all'interno dei cassonetti, a causa di diversi fattori, potrebbero smettere di funzionare. Se un sensore smette di trasmettere, il sistema riceverà dati obsoleti. Di conseguenza, i cittadini vedrebbero sulla mappa cassonetti liberi che in realtà sono già saturi, mentre la pianificazione dinamica del percorso dei mezzi prevederebbe passaggi non necessari su cassonetti vuoti o salterebbe cassonetti pieni.
  Soluzione: Oltre ai dati sulla capienza rimanente del cassonetto, ogni sensore invierà periodicamente anche lo stato di carica della batteria. Il sistema terrà traccia dell'orario degli ultimi dati ricevuti dai sensori e questo verrà mostrato sia agli utenti che agli operatori tramite una nota "aggiornato alle ore: hh:mm". Se un sensore smette di funzionare e passano più di 12 ore dall'ultimo aggiornamento ricevuto o se la carica scende al di sotto del 5%, viene aperta automaticamente una segnalazione.

- **R5 - Rischio di escludere anziani e ipovedenti**
  Descrizione: È essenziale che l'applicazione venga utilizzata dall'intera cittadinanza affinché venga effettuata la corretta raccolta dei rifiuti e avvenga la corretta comunicazione tra cittadini e amministrazione. Uno dei rischi principali è sviluppare un'interfaccia utente che risulti inaccessibile o complessa per anziani, persone ipovedenti o poco avvezze alla tecnologia. Ciò comporterebbe un basso tasso di adozione dell'app, il che vanificherebbe l'intero scopo dell'app.
  Soluzione: Sviluppo della GUI secondo le best practices suggerite dalla WCAG (Web Content Accessibility Guidelines), implementazione del supporto nativo agli screen reader, aggiunta di opzioni di ridimensionamento dei font e dei pulsanti e opzioni di aumento del contrasto dei colori.

C. Stato dell'Arte (D1)

Analizzate almeno 3 sistemi esistenti, descrivendone le funzionalità, ed evidenziando pro e contro, e quello che vorrete riutilizzare (SOTA).

Cosa verrà valutato: numero di sistemi considerati, qualità della descrizione dei sistemi esistenti, qualità dell'analisi dei pro e contro delle applicazioni, descrizione e motivazione dettagliata del perché si vuole riutilizzare quel requisito.

Junker raccolta differenziata

Junker è una applicazione Mobile per smartphone che aiuta i cittadini a differenziare in maniera corretta e rapida i rifiuti domestici.
Scansionando i codici a barre stampigliati sugli imballaggi o scrivendo la tipologia di rifiuto, l'app riconosce il prodotto, lo scompone nei materiali che lo costituiscono così da poter smistare il rifiuto con facilità in base alla raccolta differenziata adottata nel proprio comune.
L'app fornisce informazioni puntuali sulle modalità di smistamento e permette di richiedere feedback su come smaltire i prodotti più difficili da identificare o meno comuni.

PRO	CONTRO
Eccellente sistema di scansione tramite codice a barre	Non funziona in assenza di rete internet
Database di prodotti immenso e costantemente aggiornato	Manca di una piattaforma gestionale per gli operatori ecologici
Interfaccia pulita, colorata e intuitiva, adatta a un vasto pubblico	
Traduzione automatica in molteplici lingue	

- **SOTA1: Interfaccia del calendario**
  Descrizione: L'app mostra in modo visivamente chiaro e immediato quali rifiuti devono essere esposti nella giornata odierna e in quella successiva, grazie all'utilizzo di icone colorate corrispondenti al colore dei bidoni.
  Cosa si vuole riutilizzare: Si vuole riutilizzare il concetto di design dell'interfaccia utente basato sul color-coding per le tipologie di rifiuti (es. giallo per plastica, blu per carta) per la visualizzazione del calendario giornaliero e settimanale, allo scopo di rendere la lettura del calendario più rapida e intuitiva. (R5, RF2)

Il Rifiutologo

Applicazione per servizi ambientali per i cittadini dei comuni serviti dal Gruppo Hera. L'app permette di consultare il calendario della raccolta, ricevere notifiche di promemoria, cercare come smaltire un rifiuto tramite ricerca testuale, trovare la stazione ecologica più vicina tramite geolocalizzazione, prenotare un ritiro a domicilio di rifiuti ingombranti direttamente in app e inviare segnalazioni geolocalizzate con foto per comunicare rifiuti abbandonati o cassonetti danneggiati.

PRO	CONTRO
Integrazione diretta con i sistemi dell'azienda di raccolta	Interfaccia utente visivamente un po' datata rispetto ai competitor
Mappa interattiva molto dettagliata per i punti di raccolta speciali (es. pile, farmaci)	Rischio di segnalazioni falsate o non attuali (permette di caricare foto dalla galleria)
Flusso di segnalazione fotografica e GPS molto rapida	Non offusca automaticamente i dati sensibili nelle foto caricate

- **SOTA2: Mappa Interattiva e Filtrabile dei Centri di Raccolta**
  Descrizione: L'app permette di visualizzare una mappa con dei "pin" che indicano la posizione esatta di isole ecologiche e contenitori stradali per rifiuti speciali. L'utente può attivare o disattivare dei filtri per vedere, ad esempio, solo i centri di raccolta per i rifiuti elettronici, solo i contenitori per gli abiti usati o solo i raccoglitori per pile scariche e farmaci scaduti.
  Cosa si vuole riutilizzare: Si vuole riutilizzare il concetto di mappa interattiva filtrabile. Vogliamo replicare la possibilità per l'utente di selezionare una categoria specifica per vedere sulla mappa solo i punti di interesse pertinenti, migliorando l'usabilità rispetto a una mappa satura di informazioni. (R5)

TrashOut

TrashOut è un progetto ambientale che mira a mappare e monitorare tutte le discariche illegali nel mondo e informare le autorità locali.
Gli utenti utilizzano l'app per segnalare discariche illegali scattando foto, aggiungendo dettagli sulle dimensioni e sul tipo di rifiuto, e allegando la geolocalizzazione. I dati vengono poi resi disponibili su una mappa globale consultabile da enti e comuni, che possono prendere in carico la segnalazione, pulire l'area e aggiornare lo stato.
Tramite l'app si può partecipare agli eventi di pulizia, leggere articoli e vedere statistiche sul proprio paese.

PRO	CONTRO
Forte orientamento alla comunità e alla partecipazione civica (badge per gli utenti più attivi)	Non offusca automaticamente i dati sensibili nelle foto caricate
Mostrare le segnalazioni già esistenti aiuta a ridurre la ridondanza	
Ottima gestione del ciclo di vita della segnalazione lato amministratore (aperta, in lavorazione, risolta)	

- **SOTA3: Flusso di Segnalazione e Gestione Stato**
  Descrizione: L'app offre all'amministrazione una dashboard per prendere in carico e risolvere la segnalazione, tenendone traccia con cambi di stato in base all'avanzamento.
  Cosa si vuole riutilizzare: Riutilizzeremo la tracciabilità di una segnalazione tramite avanzamento di stato, implementando i seguenti stati: "Aperta", "In lavorazione", "Chiusa". Dalla dashboard degli operatori sarà possibile aggiornare lo stato di una segnalazione in base all'avanzamento, così da mantenere aggiornati gli utenti. (RF9)

D. Raffinamento dei Requisiti

A partire dai servizi minimali richiesti, raffinate la descrizione dei servizi offerti dal vostro applicativo. Descrivete anche i requisiti non funzionali.

D.1 Servizi (con prioritizzazione) (D1)

*Descrivete in **dettaglio** i servizi offerti dal vostro Sistema. In questa fase, non fate riferimento ad alcuna tecnologia specifica. Dovete acquisire una conoscenza avanzata dei problemi associati ai vostri servizi. Assegnate un ID (RF1, RF2, …) a ciascun servizio. Prioritizzate inoltre i servizi in base a due scale: importanza alta, media, bassa. Complessità alta, media, bassa. Fornire una tabella riassuntiva di importanza e complessità.*

Cosa verrà valutato: qualità dei requisiti per completezza, coerenza, tracciabilità e motivazione

- **RF1: Autenticazione utente**
  Descrizione: Il sistema deve permettere ai cittadini e agli operatori di effettuare il login e accedere al proprio profilo. Per i cittadini, l'accesso deve essere vincolato tramite l'uso di sistemi di verifica identità digitale.
  Tracciabilità: R1, RNF1.

- **RF2: Visualizzazione calendario e ricerca rifiuti**
  Descrizione: L'applicazione deve mostrare i giorni di raccolta porta a porta e includere un database testuale ricercabile per indicare all'utente in quale bidone gettare uno specifico rifiuto.
  Tracciabilità: SOTA1, R5.

- **RF3: Visualizzazione mappa centri di raccolta**
  Descrizione: L'app deve mostrare una mappa interattiva in cui l'utente può visualizzare e filtrare per categoria (es. ingombranti, farmaci) le isole ecologiche e i cassonetti speciali.
  Tracciabilità: SOTA2, R5.

- **RF4: Invio segnalazione con foto geolocalizzata**
  Descrizione: Il sistema deve permettere all'utente di segnalare un disservizio o un abbandono di rifiuti scattando obbligatoriamente una foto tramite la fotocamera in-app e allegando le coordinate GPS del dispositivo.
  Tracciabilità: C2, R3, RNF5.

- **RF5: Prevenzione segnalazioni duplicate**
  Descrizione: Prima dell'invio di una nuova segnalazione, l'app deve interrogare il database e mostrare all'utente le segnalazioni aperte nel raggio di 50 metri, chiedendo se desidera confermare una di quelle esistenti o procedere con la creazione di una nuova.
  Tracciabilità: C2.

- **RF6: Oscuramento dati sensibili con IA**
  Descrizione: Il backend deve processare ogni foto ricevuta tramite un modello IA di object-detection per sfocare automaticamente volti e targhe prima del salvataggio nel database.
  Tracciabilità: R2, RNF2.

- **RF7: Acquisizione telemetria sensori**
  Descrizione: l server deve ricevere e registrare periodicamente i dati provenienti dai sensori nei cassonetti, come percentuale di riempimento e livello di batteria, e deve tenere traccia dell'orario di ricezione di tali dati.
  Tracciabilità: R4, RNF4.

- **RF8: Calcolo percorsi ottimali di raccolta**
  Descrizione: Il sistema deve inviare i dati validati (sensori e segnalazioni) ad un'API esterna per generare il percorso ottimizzato per i camion, basandosi su capienza dei mezzi e orari.
  Tracciabilità: C1.

- **RF9: Dashboard operatori e gestione stato segnalazioni**
  Descrizione: Il sistema deve fornire un'interfaccia web agli operatori per visualizzare dati aggregati e statistiche, i percorsi calcolati e per aggiornare lo stato delle segnalazioni (Aperta, In lavorazione, Chiusa).
  Tracciabilità: SOTA3.

- **RF10: Invio notifiche push e promemoria**
  Descrizione: Il sistema lato backend deve inviare notifiche push automatiche all'applicazione mobile dell'utente per ricordare l'esposizione del bidone corretto la sera precedente alla raccolta, e consentire agli operatori comunali di inviare avvisi straordinari (es. scioperi, modifiche alla viabilità) dalla dashboard.
  Tracciabilità: RF2.

- **RF11: Download delle mappe offline**
  Descrizione: L'applicazione mobile deve permettere all'utente di scaricare e salvare in memoria porzioni di mappa e le posizioni dei centri di raccolta, permettendone la consultazione anche in assenza di copertura di rete.
  Tracciabilità: R3, RNF4.

- **RF12: Limitazione frequenza segnalazioni**
  Descrizione: Il sistema lato backend deve imporre un tempo di attesa (timeout) di 15 minuti tra l'invio di una segnalazione e la successiva da parte dello stesso account, rifiutando richieste multiple troppo ravvicinate.
  Tracciabilità: R1, RF4.

ID Servizio	Nome Servizio	Priorità	Complessità
RF1	Autenticazione Utente	ALTA	MEDIA
RF2	Calendario e ricerca rifiuti	ALTA	BASSA
RF3	Mappa centri di raccolta	MEDIA	BASSA
RF4	Creazione segnalazione geolocalizzata	ALTA	MEDIA
RF5	Prevenzione segnalazioni duplicate	BASSA	MEDIA
RF6	Elaborazione IA oscuramento	MEDIA	ALTA
RF7	Acquisizione telemetria sensori	ALTA	MEDIA
RF8	Calcolo percorsi di raccolta	ALTA	ALTA
RF9	Dashboard operatori e gestione stato	MEDIA	MEDIA
RF10	Invio notifiche push e promemoria	MEDIA	MEDIA
RF11	Download delle mappe offline	BASSA	ALTA
RF12	Limitazione frequenza segnalazioni	BASSA	BASSA

D.2 Use Case Diagrams (D1)

Riportare in questa sezione gli use case diagram relativi al vostro sistema. Associare ad ogni use case diagram un ID (UCD1, UDC2, …). Se necessario, inserire anche la descrizione degli attori non riportati nella sezione precedente. Creare un use case diagram dell'intero sistema. Aggiungete altri use case diagram se lo ritenete opportuno.

Cosa verrà valutato: coerenza, completezza, correttezza sintattica e semantica, tracciabilità con requisiti e scenari.

- **A1: Cittadino**
  Descrizione: Utente finale dell'applicazione mobile. È il residente del comune che utilizza l'app per consultare il calendario della raccolta, trovare i centri ecologici e inviare segnalazioni di disservizi o abbandoni sul territorio.

- **A2: Operatore Ecologico**
  Descrizione: Utente finale della dashboard web. Gestisce l'avanzamento di stato delle segnalazioni inviate dai cittadini, visualizza le statistiche e avvia la generazione dei percorsi ottimizzati per i mezzi di raccolta.

- **A3: Sistema SPID/CIE**
  Descrizione: Attore di sistema esterno. Si occupa di verificare e validare l'identità del cittadino durante la fase di login. Serve a garantire sicurezza nell'autenticazione e prevenire la creazione di account falsi o bot.

- **A4: Sensore Cassonetto**
  Descrizione: Attore di sistema hardware. Trasmette autonomamente al server i dati di telemetria (livello di riempimento e stato della batteria) a intervalli regolari, necessario per avere dati sempre aggiornati e garantire tempestività nella raccolta rifiuti.

- **A5: API Routing**
  Descrizione: Attore di sistema esterno. Riceve le coordinate e i dati sui carichi dal backend ed elabora matematicamente il percorso stradale ottimizzato per i mezzi di raccolta. Essenziale per garantire affidabilità ed efficienza per il calcolo.

- **UCD1: Sottosistema App Mobile (Lato Cittadino)**

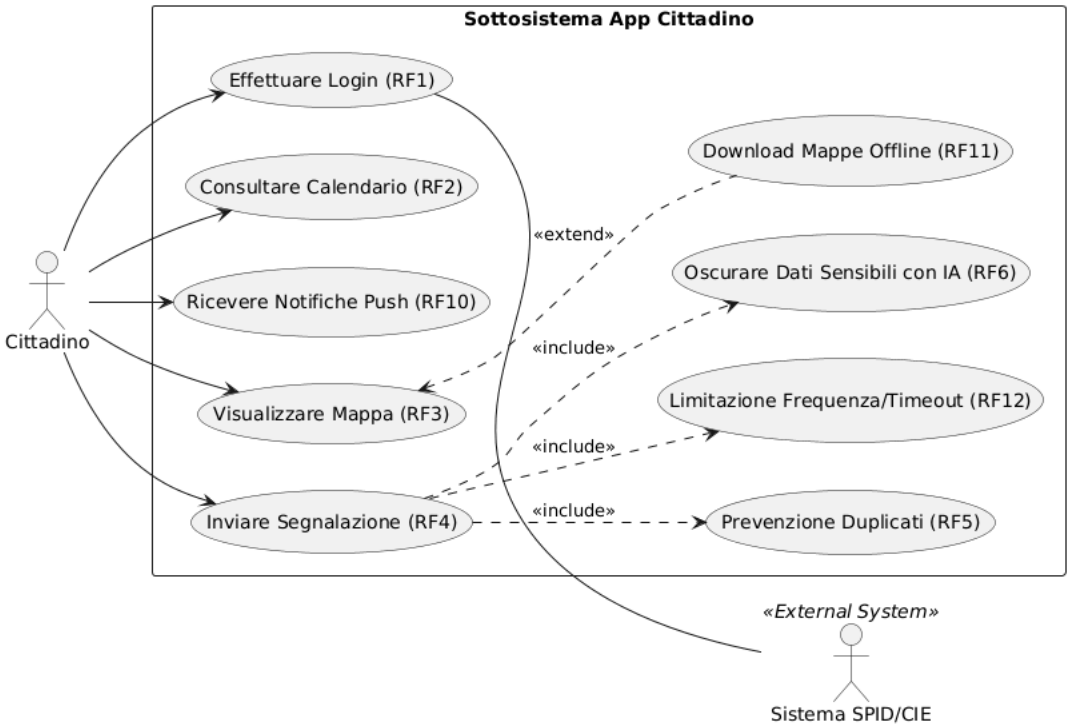


Diagramma realizzato in PlantUML e Draw.io.

Descrizione: Il diagramma illustra le interazioni del cittadino con l'App Mobile.
Il cittadino può: effettuare il login, consultare il calendario della raccolta, ricevere notifiche push, visualizzare la mappa con i punti di raccolta e inviare segnalazioni.
Nel diagramma vengono messi in evidenza i seguenti punti:

- Il login viene delegato al sistema esterno di SPID/CIE;
- Il sistema attiva automaticamente dei meccanismi durante l'invio di una segnalazione (l'offuscamento IA, il controllo dei duplicati e il limitatore frequenza/timeout);
- È presente l'opzione di scaricare la mappa in locale per la visualizzazione in assenza di rete.

Tracciabilità: RF1, RF2, RF3, RF4, RF5, RF6, RF10, RF11, RF12.

- **UCD2: Sottosistema Dashboard Web e Backend (Lato Operatore)**

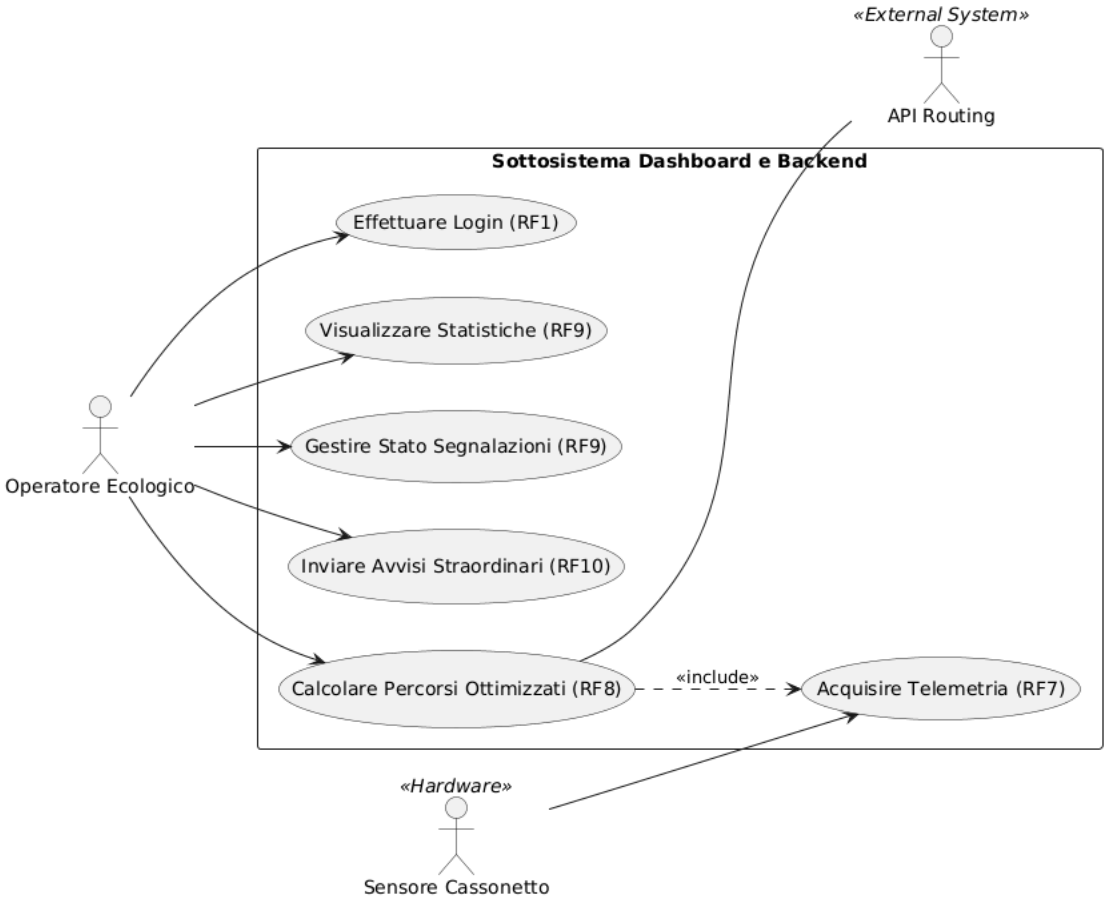


Diagramma realizzato in PlantUML e Draw.io.

Descrizione: Il diagramma illustra le interazioni dell'operatore con la dashboard gestionale e la parte logistica.
L'operatore può effettuare il login, visualizzare dati aggregati e statistiche, gestire lo stato delle segnalazioni, inviare avvisi straordinari e avviare il calcolo dei percorsi ottimizzati dei mezzi di raccolta.
Il diagramma mette in evidenza i seguenti punti della logistica:

- Il sistema delega il calcolo dei percorsi ottimizzati ad un API di Routing esterna;
- Il sistema acquisisce i dati di telemetria dei cassonetti dai sensori esterni;
- I dati dai sensori esterni vengono obbligatoriamente inclusi nel calcolo dei percorsi ottimizzati.

Tracciabilità: RF1, RF7, RF8, RF9, RF10.

D.3 Requisiti non Funzionali (D1)

Listate almeno 3 requisiti non funzionali più importanti per il vostro Sistema. Associate ad ogni requisito non funzionale un ID (RNF1, RNF2…). Per ogni requisito riportarne la descrizione, motivare il perchè il requisito è fondamentale per il sistema (cosa può succedere se non soddisfatto), e tracciabilità con altri requisiti/rischi/challenges.
Cosa verrà valutato: descrizione completa, coerenza con traccia/contesto, misurabilità, motivazione.

- **RNF1: Sicurezza dell'Identità Digitale**
  Descrizione: L'autenticazione del cittadino (RF1) deve avvenire esclusivamente tramite protocollo SPID o CIE.
  Motivazione: È fondamentale per garantire la reale identità di chi invia segnalazioni, prevenendo la creazione di bot e mitigando il rischio di attacchi spam al database comunale.
  Tracciabilità: RF1, R1.

- **RNF2: Conformità Privacy (GDPR)**
  Descrizione: Nessuna foto con dati sensibili, volti o targhe di passanti deve essere salvato in chiaro nei server.
  Motivazione: Requisito legale della tutela della privacy dei consumatori. Se non soddisfatto, il comune rischia sanzioni per violazione del GDPR.
  Tracciabilità: RF6, R2.

- **RNF3: Accessibilità Inclusiva**
  Descrizione: Le interfacce devono avere contrasto elevato, font scalabili, supporto a screen reader, per rispettare le direttive WCAG.
  Motivazione: L'inaccessibilità per anziani o ipovedenti causerebbe un basso tasso di adozione del sistema, vanificando l'investimento del Comune sulla corretta raccolta dei rifiuti.
  Tracciabilità: RF2, R5.

- **RNF4: Tolleranza ai guasti dei sensori**
  Descrizione: Il backend deve identificare e isolare autonomamente i dati provenienti da sensori hardware malfunzionanti, ignorando qualsiasi lettura proveniente da un sensore che non invia aggiornamenti da più di 12 ore o che ha un livello di carica inferiore al 5%.
  Motivazione: Essenziale per garantire l'integrità dei dati usati per il calcolo dei percorsi ottimali di raccolta (RF8). Se il sistema non fosse in grado di tollerare il guasto di un sensore, l'algoritmo di routing riceverebbe dati obsoleti causando inefficienze nella pianificazione dei percorsi dei mezzi.
  Tracciabilità: RF7, RF8, R4.

- **RNF5: Disponibilità e Funzionamento Offline**
  Descrizione: L'applicazione mobile deve garantire una disponibilità continua delle funzionalità primarie (visualizzazione mappa salvata e inserimento dati per nuove segnalazioni) anche in totale assenza di copertura di rete, salvando le operazioni in cache locale per sincronizzarle al ripristino del segnale.
  Motivazione: Fondamentale per la continuità operativa del cittadino sul territorio. Senza questo vincolo, l'utente sarebbe impossibilitato a segnalare disservizi o abbandoni nelle aree del comune con scarsa copertura cellulare, causando frustrazione e abbandono dell'app.
  Tracciabilità: RF4, RF11, R3.

- **RNF6: Efficienza e Tempi di Latenza**
  Descrizione: Il sistema deve garantire tempi di risposta rapidi per non degradare la User Experience. L'elaborazione e l'oscuramento IA delle immagini lato server (RF6) deve concludersi entro 10 secondi dalla ricezione. Il calcolo del routing dinamico (RF8) dei mezzi di raccolta deve essere generato in meno di 10 minuti.
  Motivazione: Tempi di attesa troppo lunghi nell'invio della segnalazione porterebbero l'utente a chiudere l'app pensando che ci sia stato un blocco del sistema. Lato operatori, un algoritmo di routing troppo lento potrebbe intralciare o rallentare l'operatività e la tempestività dei mezzi.
  Tracciabilità: RF6, RF8.

- **RNF7: Compatibilità Multi-Piattaforma**
  Descrizione: L'applicazione mobile (lato cittadino) deve essere sviluppata per essere compatibile e scaricabile sia su sistemi operativi Android (versione 6.0 o superiore) che iOS (versione 15.5 o superiore).
  Motivazione: Essendo un servizio pubblico rivolto all'intera cittadinanza, il software dovrebbe poter essere scaricato da tutti, a prescindere dalla marca dello smartphone, e supportare quindi anche versioni più vecchie dei sistemi operativi. Le persone meno avvezze alla tecnologia spesso tendono a non mantenere i loro smartphone aggiornati alle ultime versioni software.
  Tracciabilità: R5.

E. Architettura Software

Riportare qui la struttura statica e dinamica del vostro sistema in termini di component, sequence e state diagrams.

E.1 Vista statica del sistema: Component Diagram (D2.A)

Inserire in questa sezione Il component diagram relativo al vostro sistema. Descriverne ogni componente e riportare la tracciabilità con requisiti funzionali e non funzionali, se necessario con challenges e rischi.

Cosa verrà valutato: correttezza sintattica e semantica, documentazione, tracciabilità.

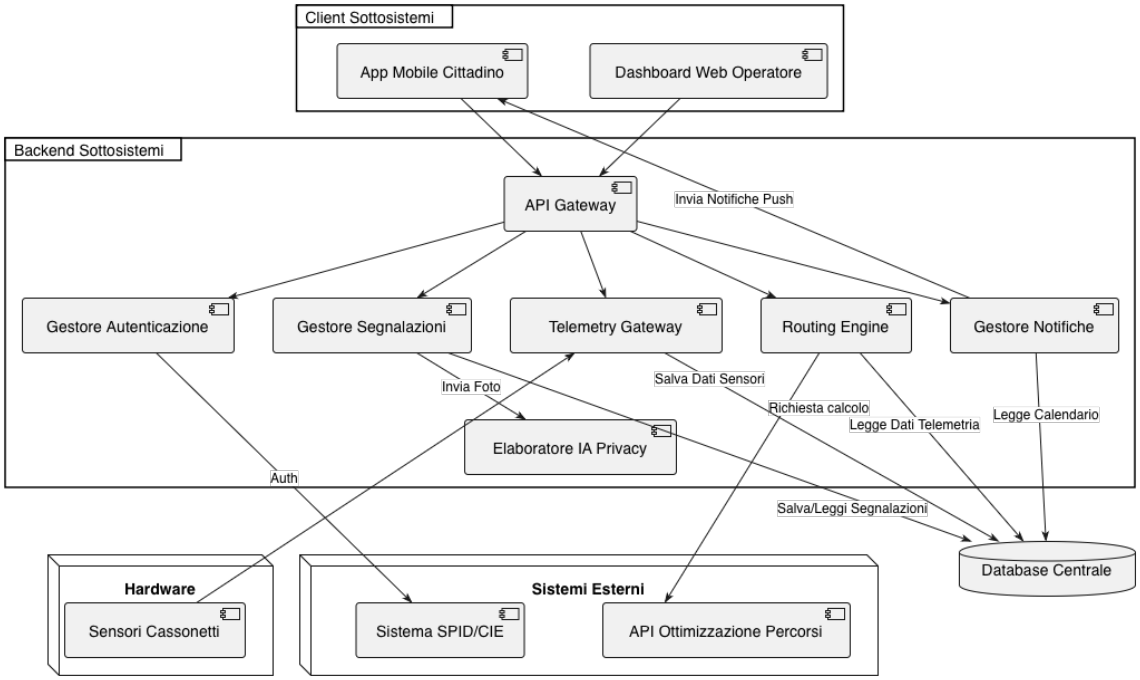


Diagramma realizzato in PlantUML e Draw.io.

Descrizione testuale dell'architettura:

- **Comp1: App Mobile Cittadino**
  Descrizione: Applicazione frontend per i cittadini. Permette la consultazione del calendario della raccolta, la visualizzazione della mappa filtrabile e l'invio di segnalazioni geolocalizzate con foto. Include un modulo di caching per scaricare le mappe e salvare le segnalazioni create offline, inviandole al ripristino della rete.
  Tracciabilità: RF2, RF3, RF4, RF10, RF11, RNF3, RNF5, RNF7, R3, R5.

- **Comp2: Dashboard Web Operatore**
  Descrizione: Interfaccia web dedicata agli operatori ecologici comunali. Viene utilizzata per visualizzare statistiche aggregate, gestire lo stato di avanzamento delle segnalazioni e lanciare il calcolo dei percorsi ottimizzati.
  Tracciabilità: RF1, RF9, RF10, SOTA3.

- **Comp3: API Gateway**
  Descrizione: Punto di accesso unificato per tutte le richieste provenienti dai client verso il backend. Smista il traffico verso i vari microservizi, in questo modo il lato client non avrà mai accesso diretto alle singole componenti backend.
  Tracciabilità: RNF6, RF1, RF4, RF5, RF8, RF10

- **Comp4: Gestore Autenticazione**
  Descrizione: Modulo responsabile del login degli utenti. Si interfaccia con il sistema esterno SPID/CIE per validare l'identità dei cittadini.
  Tracciabilità: RF1, R1, RNF1.

- **Comp5: Gestore Segnalazioni**
  Descrizione: Componente che riceve e gestisce le segnalazioni. Controlla il raggio di 50 metri per verificare la presenza di segnalazioni simili e prevenire duplicati e spam, e gestisce il ciclo di vita (cambi di stato) comunicando con la dashboard.
  Tracciabilità: RF4, RF5, RF9, C2.

- **Comp6: Elaboratore IA Privacy**
  Descrizione: Modulo che riceve le foto dal Gestore Segnalazioni e, tramite un modello IA, elabora ogni foto in entrata per rilevare e sfocare automaticamente volti, targhe e dati sensibili prima del salvataggio definitivo.
  Tracciabilità: RF6, R2, RNF2, RNF6.

- **Comp7: Routing Engine**
  Descrizione: Componente per il calcolo dinamico dei percorsi. Estrae i dati sulla capienza dei cassonetti e la posizione delle segnalazioni e li invia alle API di routing esterne per ottenere i percorsi stradali ottimali.
  Tracciabilità: RF8, C1, RNF6.

- **Comp8: Telemetry Gateway**
  Descrizione: Servizio in ascolto dei dati provenienti dai sensori hardware. Riceve livello di riempimento e carica della batteria, implementando una logica di tolleranza ai guasti: apre automaticamente una segnalazione se un sensore non risponde per 12 ore o ha la batteria sotto il 5%.
  Tracciabilità: RF7, R4, RNF4.

- **Comp9: Gestore Notifiche**
  Descrizione: Componente che interroga il calendario nel Database Centrale e gestisce l'invio di promemoria automatici push, oltre a inoltrare gli avvisi straordinari creati dagli operatori.
  Tracciabilità: RF10.

- **Comp10: Database Centrale**
  Descrizione: Sistema di persistenza dei dati. Memorizza utenti, stato e foto delle segnalazioni, dati di telemetria e configurazioni del calendario.
  Tracciabilità: RF4, RF10.

- **Comp11: Sensori Cassonetti (Hardware)**
  Descrizione: Dispositivi hardware che trasmettono lo stato di riempimento in tempo reale.
  Tracciabilità: A4, R4.

- **Comp12: Sistema SPID/CIE (Esterno)**
  Descrizione: Provider di identità digitale di terze parti.
  Tracciabilità: A3, RNF1.

- **Comp13: API Ottimizzazione Percorsi (Esterno)**
  Descrizione: Servizio esterno dedicato al calcolo complesso dei tracciati.
  Tracciabilità: A5, C1.

E.2 Vista dinamica dell'architettura software: Sequence Diagram (D2.A)

Inserire in questa sezione almeno 3 sequence diagrams relativi al vostro sistema. Dopo averne riportato l'immagine, datene una descrizione testuale. Riportare per ogni sequence anche dati sulla tracciabilità con i vari componenti del component diagram.

Cosa verrà valutato: correttezza sintattica e semantica, documentazione, tracciabilità con le componenti.

- **SD1: Invio Segnalazione con Offuscamento IA e Prevenzione Duplicati**

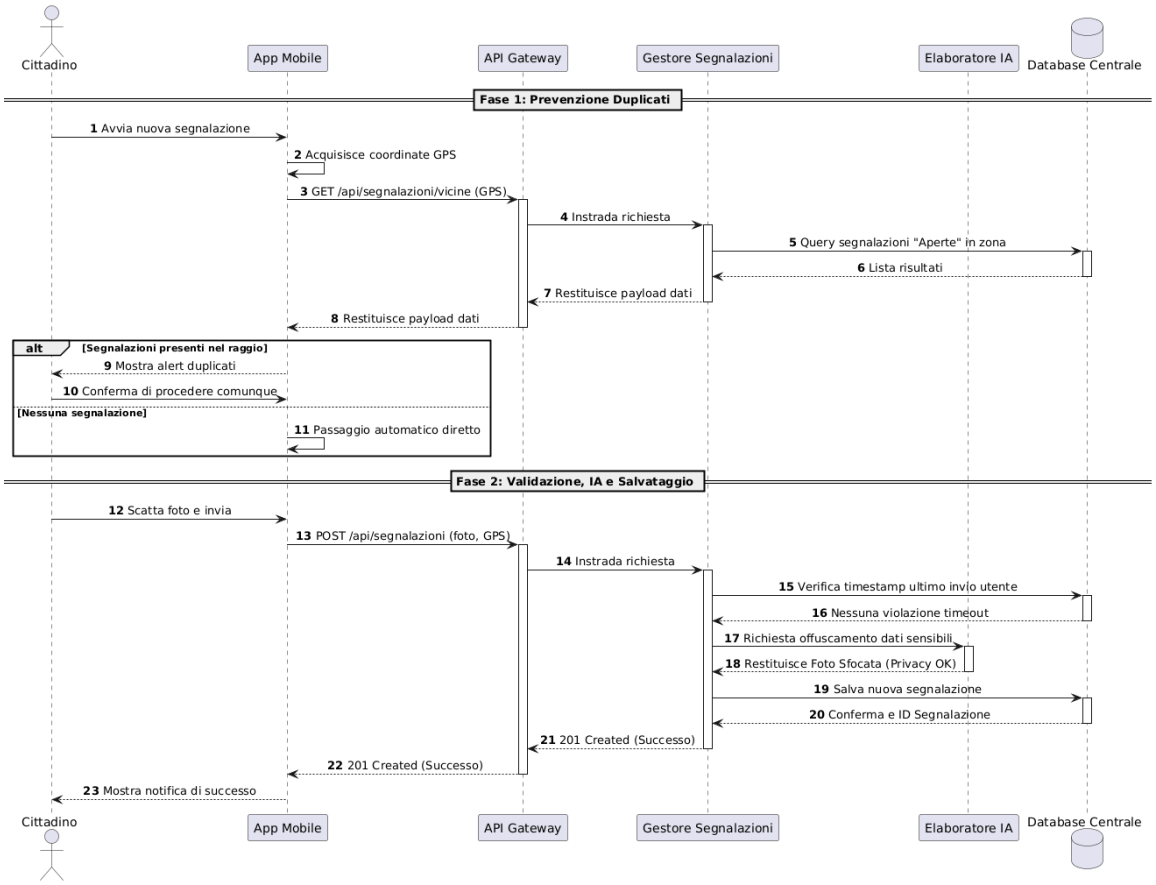


Diagramma realizzato in PlantUML e Draw.io.

Descrizione: Questo diagramma modella il flusso di invio di una segnalazione geolocalizzata da parte di un cittadino. Appena il cittadino avvia una segnalazione, l'App Mobile acquisisce la posizione del cittadino e la passa, tramite l'API Gateway al Gestore Segnalazioni. Questo interroga il Database Centrale per cercare segnalazioni già aperte nel raggio di 50 metri dalla posizione

ricevuta e in caso ce ne fosse già una aperta, la restituisce e viene mostrata al cittadino che può decidere se continuare o meno con la segnalazione. Se decide di proseguire o se non vengono trovati duplicati, il cittadino scatta la foto che viene passata al Gestore Segnalazione, qui viene effettuato il controllo per il limite di frequenza, controllando se l'utente viola il timeout di 15 minuti tra un invio e il successivo. Se il controllo è superato, l'Elaboratore IA applica (entro 10 secondi) la sfocatura su volti e targhe, la segnalazione viene salvata sul Database e il cittadino riceve una notifica di successo.
Tracciabilità: RF4, RF5, RF6, RF12, RNF2, RNF6, C2, R1, R2.

- **SD2: Calcolo Dinamico dei Percorsi di Raccolta**

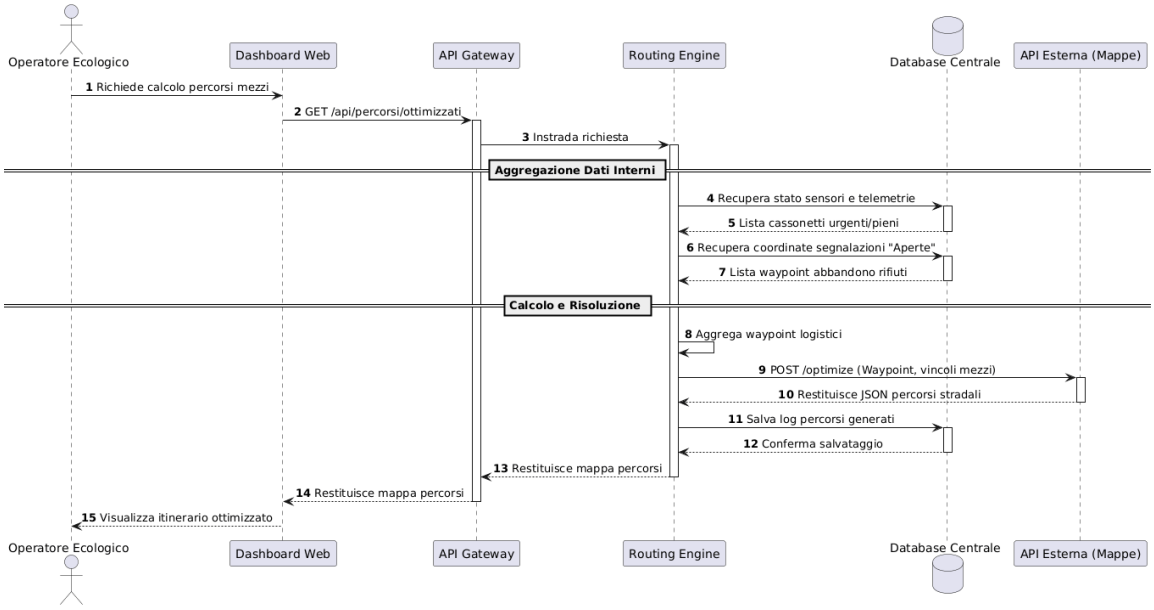


Diagramma realizzato in PlantUML e Draw.io.

Descrizione: Questo diagramma illustra il processo di generazione dei percorsi logistici ottimizzati. L'Operatore, tramite la Dashboard Web, avvia la richiesta di calcolo. Il traffico passa per l'API Gateway e arriva al Routing Engine.
Quest'ultimo interroga il Database Centrale per recuperare sia lo stato in tempo reale dei cassonetti (dati inviati dai sensori) sia le coordinate delle segnalazioni di abbandono aperte. I dati raccolti vengono passati all'API di Ottimizzazione Percorsi esterna (es. Google Maps Route Optimization), delegando il calcolo complesso dei tracciati. Una volta ottenuto il percorso ottimale (rispettando i vincoli di latenza imposti dal sistema, < 10 minuti), questo viene salvato sul Database Centrale e inviato alla Dashboard per la visualizzazione.
Tracciabilità: RF8, RF9, C1, RNF6.

- **SD3: Acquisizione Telemetria e Fault Tolerance**

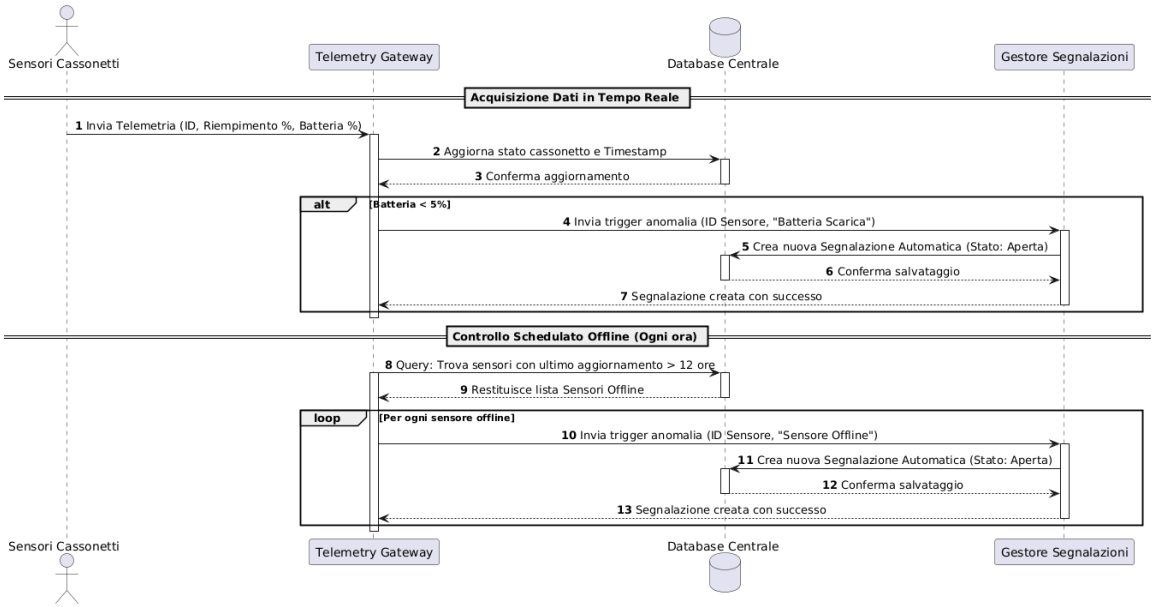


Diagramma realizzato in PlantUML e Draw.io.

Descrizione: Questo diagramma illustra il flusso di acquisizione dei dati provenienti dai sensori hardware posizionati nei cassonetti e le contromisure attivate in caso di guasto. Il processo si divide in due fasi:
Nel diagramma vengono messi in evidenza i seguenti punti:

- Ricezione standard: Il sensore invia periodicamente il payload (riempimento e batteria) al Telemetry Gateway, che aggiorna il Database Centrale registrando il timestamp dell'operazione. Se la batteria è inferiore al 5%, il Gateway contatta il Gestore Segnalazioni per aprire automaticamente un ticket di manutenzione.
- Controllo periodico: Il Telemetry Gateway esegue un controllo periodico sul Database Centrale per individuare i sensori che non trasmettono da oltre 12 ore. Per ogni sensore offline rilevato, viene generata una segnalazione automatica.

Tracciabilità: RF7, RF9, RNF4, R4.

- **SD4: Gestione Notifiche Push**

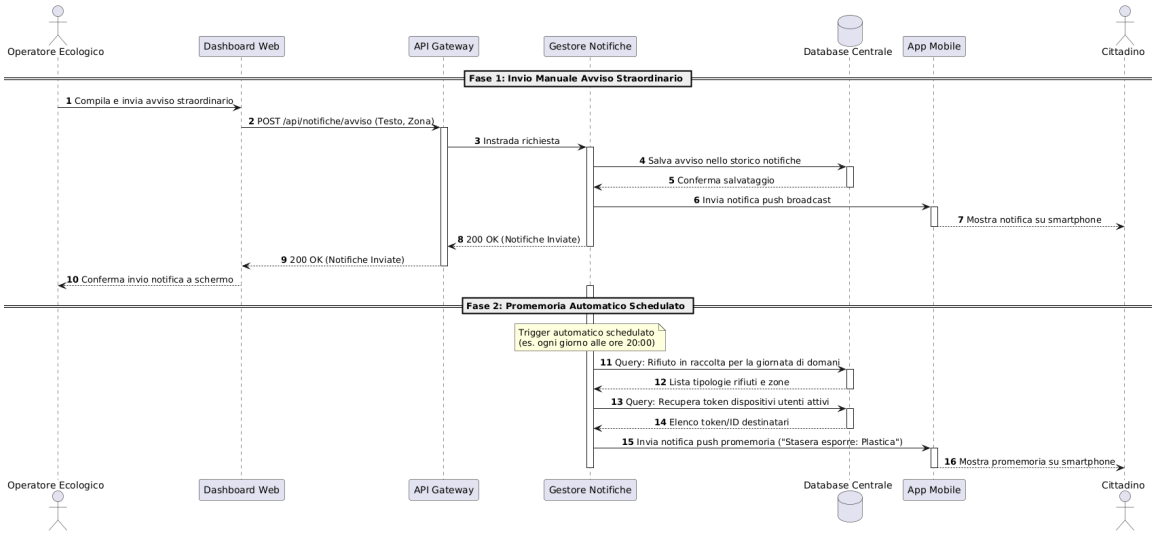


Diagramma realizzato in PlantUML e Draw.io.

Descrizione: Questo diagramma illustra il funzionamento del sistema dell'invio di notifiche push, diviso in due fasi operative distinte:
Nel diagramma vengono messi in evidenza i seguenti punti:

- Invio Manuale: L'Operatore Ecologico utilizza la Dashboard Web per inviare un avviso straordinario. La richiesta attraversa l'API Gateway fino al Gestore Notifiche, che salva lo storico nel Database Centrale ed effettua l'invio delle notifiche verso le App Mobile dei cittadini.
- Promemoria Schedulato: Un trigger di sistema automatizzato si attiva quotidianamente (es. alle ore 20:00). Il Gestore Notifiche interroga il Database Centrale per leggere dal calendario quale tipologia di rifiuto è prevista per la raccolta del giorno successivo. Successivamente, recupera i token dei dispositivi attivi e invia in automatico la notifica push di promemoria ai cittadini.

Tracciabilità: RF7, RF9, RNF4, R4.

E.3 Vista dinamica del sistema: State Diagram (D2.B)

Inserire in questa sezione lo state diagram che rappresenta il vostro sistema. Datene anche una descrizione.

Cosa verrà valutato: correttezza sintattica e semantica, documentazione.

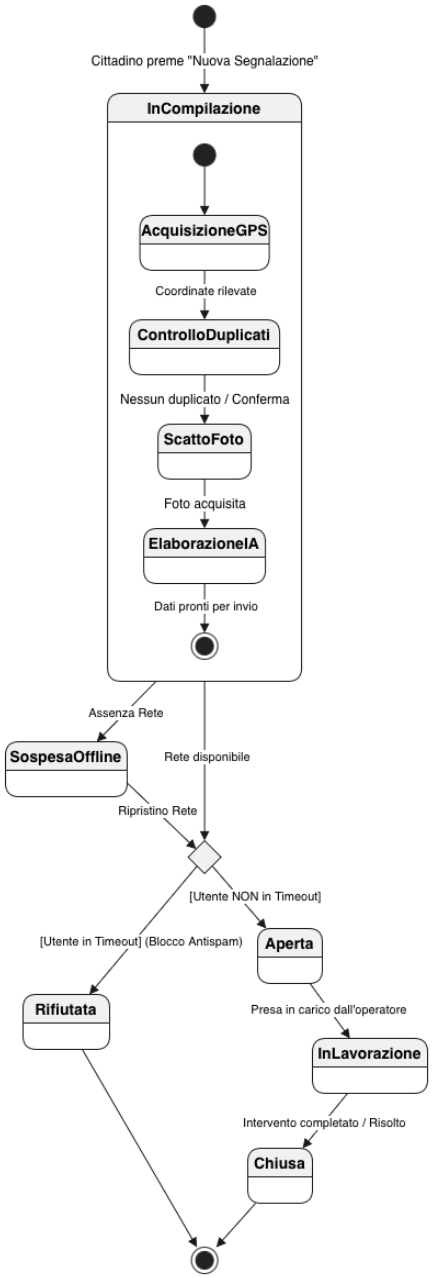


Diagramma realizzato in PlantUML e Draw.io.

Il diagramma illustra il ciclo di vita di una Segnalazione, dalle fasi di creazione tramite App Mobile alla gestione della connettività, per poi seguire i cambi di stato.

- In Compilazione: Quando il cittadino avvia una segnalazione, il sistema acquisisce le coordinate GPS, controlla gli eventuali duplicati nel raggio di 50 metri, fa scattare la foto in-app e successivamente la elabora tramite intelligenza artificiale per oscurare i dati sensibili.
- Sospesa Offline: Se al momento dell'invio lo smartphone non ha copertura di rete, la segnalazione non va persa ma viene salvata nella cache locale. Verrà spostata automaticamente nello stato “Aperta” non appena il sistema avrà di nuovo connettività.
- Aperta: La segnalazione è stata ricevuta dal backend ed è visibile sulla Dashboard dell'operatore.
- In Lavorazione: L'operatore ecologico ha preso in carico il ticket e sta pianificando o eseguendo l'intervento sul territorio.
- Chiusa / Rifiutata (Stati Finali): Se l'intervento ha successo, la segnalazione passa allo stato finale “Chiusa”. Se invece l’utente è in timeout, la segnalazione non passa il blocco antispam e questa viene portata nello stato “Rifiutata” per chiudere il ciclo.

F. Design di basso livello *(D2.B)*

Riportate in questa sezione il class diagram relativo al vostro sistema. Date una descrizione di ogni classe ponendo attenzione alla tracciabilità.

Cosa verrà valutato: correttezza sintattica e semantica, documentazione.

F.1 Class Diagram

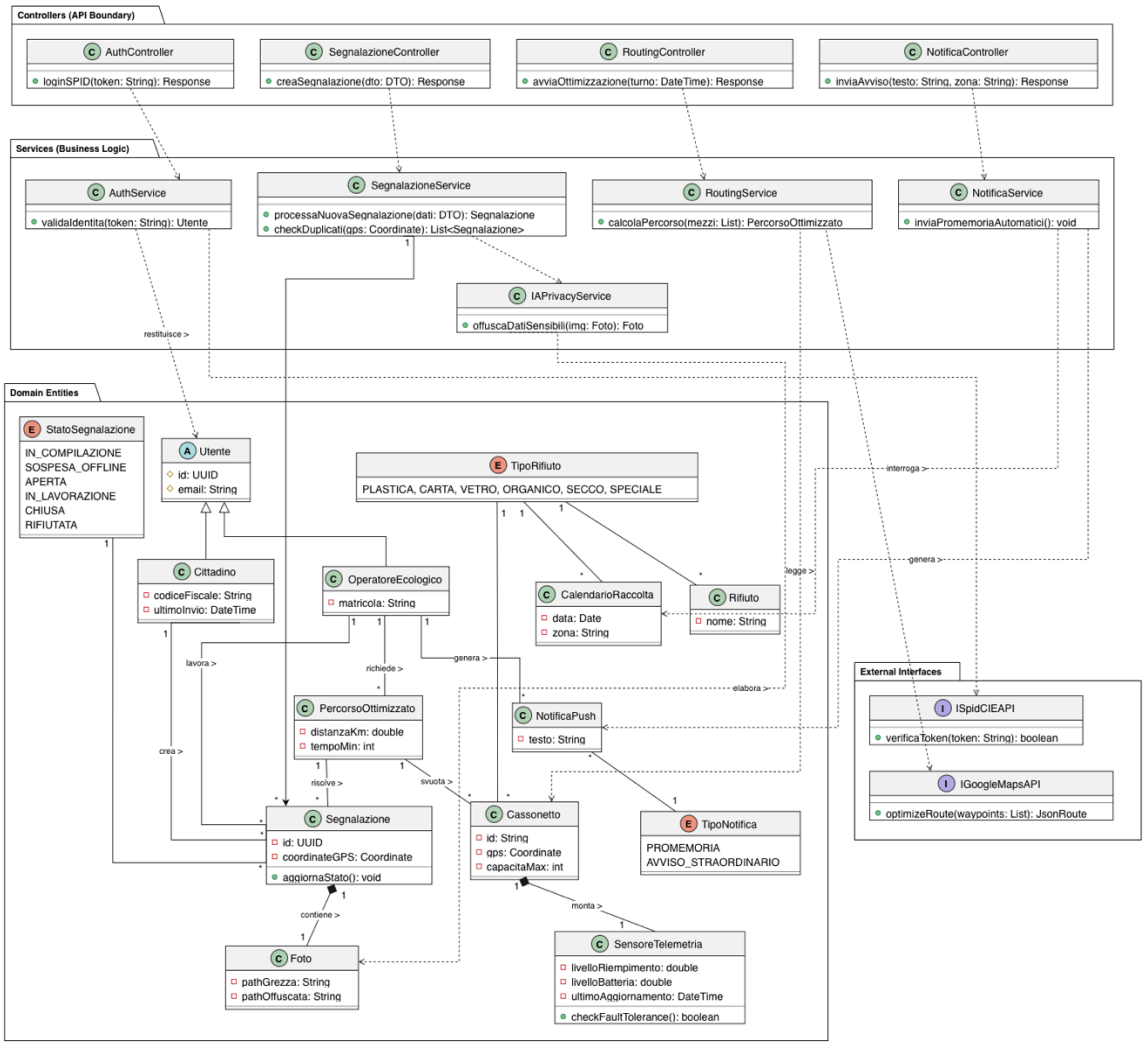


Diagramma realizzato in PlantUML e Draw.io.

Il class diagram illustra il design di basso livello dell'architettura software, suddiviso in layer per garantire modularità, scalabilità e netta separazione tra logica di business e infrastruttura.

Sono presenti i seguenti layer:

- Layer Controllers (API Boundary): Intercettano le richieste in ingresso dai client. Troviamo l'AuthController per il login (RF1, RNF1), il SegnalazioneController (RF4, RF5, R1) per la ricezione delle nuove segnalazioni dai cittadini, e i controller RoutingController (RF8, RNF6, C1) e NotificaController (RF10) per l'avvio del calcolo del percorso ottimale e l'invio di avvisi, entrambe invocate dalla dashboard degli operatori.
- Layer Services (Business Logic): Contengono la logica applicativa pura. L'AuthService valida l'identità dell'utente per il login (RF1, RNF1); Il SegnalazioneService implementa il controllo dei duplicati tramite coordinate GPS (RF12, C2); l'IAPrivacyService elabora le foto garantendo la conformità alla Privacy (RNF2, R2); il RoutingService legge lo stato dei sensori e coordina il calcolo logistico (RF8, RNF6, C1); il NotificaService (RF10) gestisce l'invio di notifiche push asincrono.
- Layer Domain Entities: Rappresenta il modello concettuale e le regole di business. Include l'ereditarietà degli utenti, implementata con una classe astratta Utente estesa da Cittadino (A1) e OperatoreEcologico (A2); l'entità principale Segnalazione, con composizione rigida verso Foto; e Cassonetto composto da SensoreTelemetria, gli elementi per la raccolta di dati dai sensori (R4). StatoSegnalazione, TipoRifiuto e TipoNotifica sono rappresentati da tipi enumerativi.
- Layer External Interfaces: Contiene le API di terze parti ISpidCIEAPI (R1, RF1, RNF1) e IGoogleMapsAPI (RF8, RNF6, C1) per separare i servizi interni da quelli esterni.

Appendix A. Effort Recording

(D1, D2.A, D2.B, D3)

Durante lo svolgimento dell'assegnazione, ***registra ciò che stai facendo e quanto tempo hai impiegato****. Come regola generale, dovresti aggiungere una nuova voce di registro ogni volta che cambi attività. Ad esempio, se lavori in modo continuativo su un'unica attività per due ore, puoi inserire una sola voce. Tuttavia, se in mezz'ora svolgi due o tre attività differenti, devi inserire una voce separata per ciascuna di esse.*

Non è necessario includere il tempo impiegato per la compilazione del registro stesso, ma devi includere il tempo trascorso nello svolgimento delle altre parti del compito — ***incluso, se e quando applicabile, quello dedicato all'interazione con il modello LLM****.*

Quando registri il tempo speso per il progetto, crea diverse ***sottocategorie****. È importante distinguere chiaramente tra due categorie principali:*

- ***Learning (Apprendimento)*** *– il tempo impiegato per studiare i linguaggi di modellazione, gli strumenti, o — se previsto — per comprendere come utilizzare un modello LLM. Un esempio dei task da considerare: Studio Requisiti, Studio Requisiti Non Funzionali, Studio Use Case Diagrams, Studio Component Diagrams, Studio Sequence Diagrams, Studio State Diagrams.*

- ***Doing (Svolgimento)*** *– il tempo dedicato alla creazione dei modelli, alla definizione dei requisiti, alla presa di decisioni o alla revisione del lavoro svolto (anche con eventuale supporto dell'LLM). Un esempio dei task da considerare: Scoperta dei requisiti, analisi dello Stato dell'arte, analisi dei Rischi, Modellazione Use Case Diagrams, Modellazione Component Diagrams, Modellazione Sequence Diagram, Modellazione State Diagram.*

Si riportino tali informazioni nell'excel fornito insieme a questa template.

Troverai un foglio riassuntivo per tutto il team, ed un foglio per ciascun membro del team. Va consegnato un file Excel completo per ciascun deliverable di progetto.